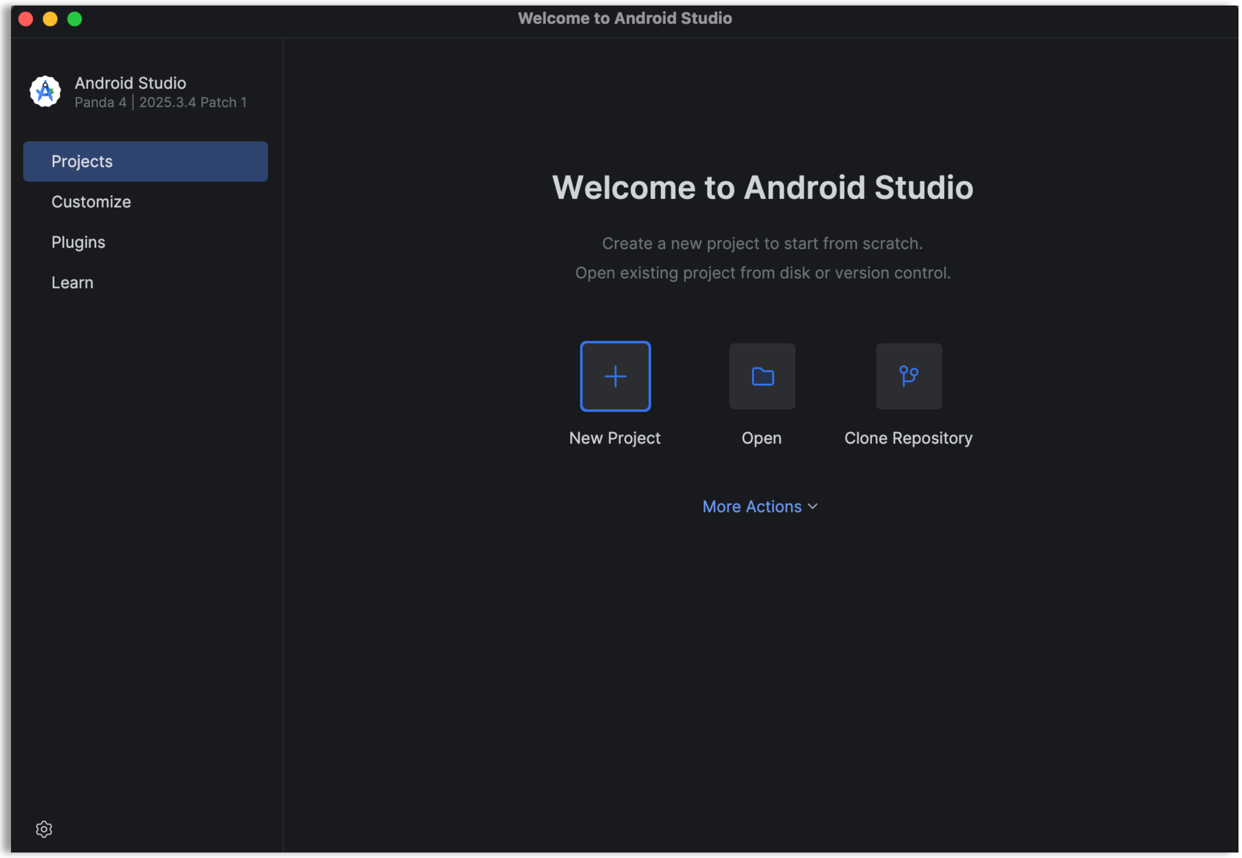
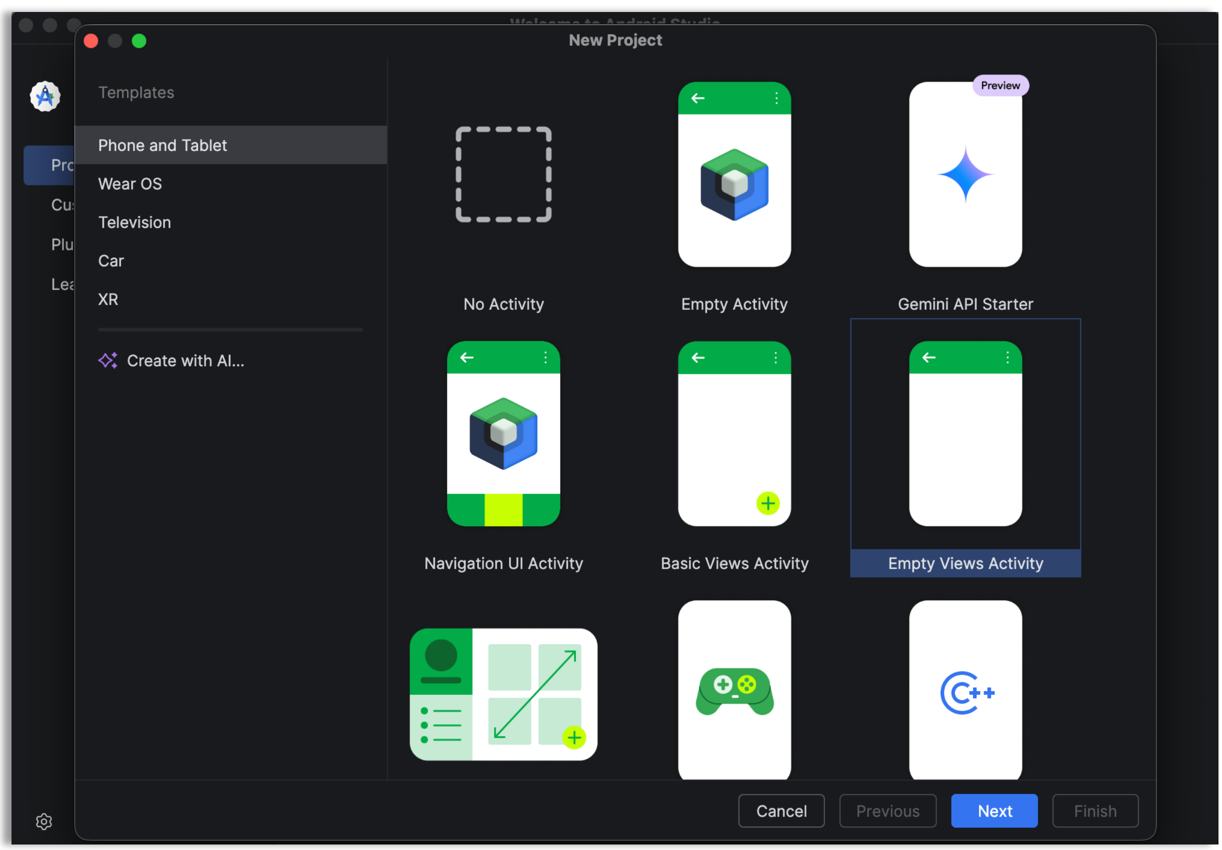
ListyCity – Instructions

1. Create a new project in Android Studio as shown below by clicking on 'New Project'

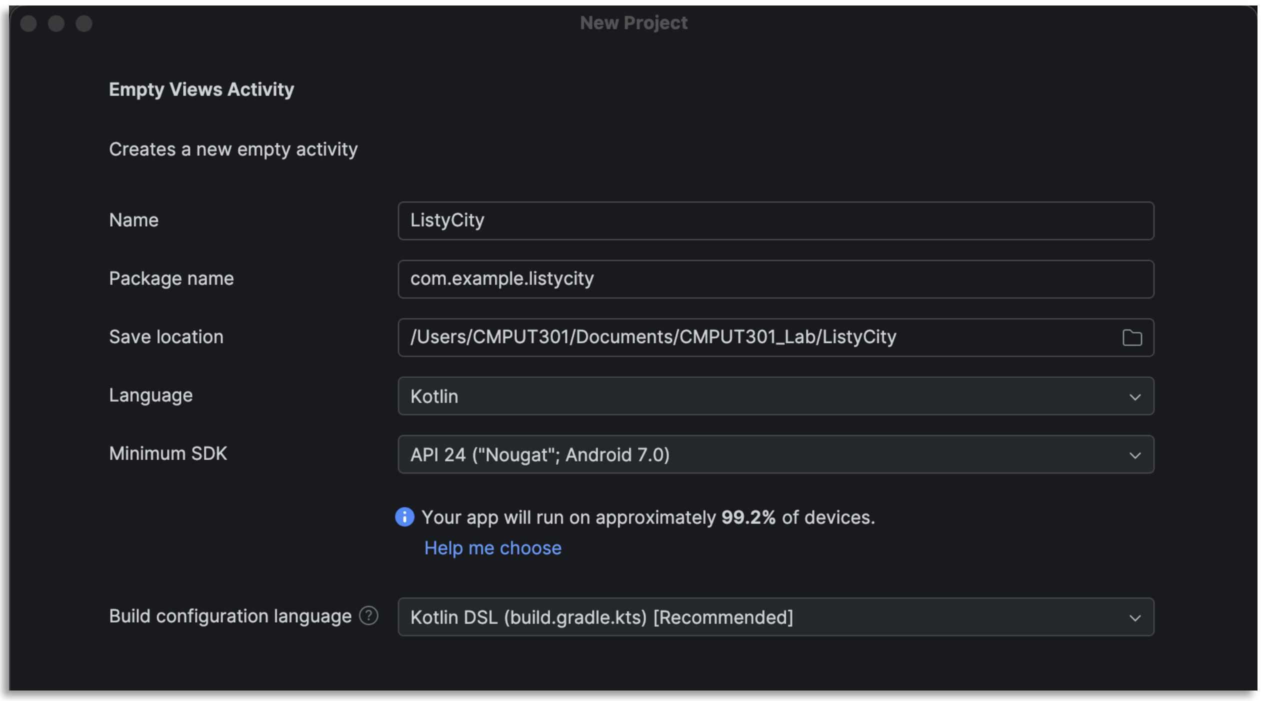


2. Select 'Empty Views Activity' under the 'Phone and Tablet' tab.
   Click Next.

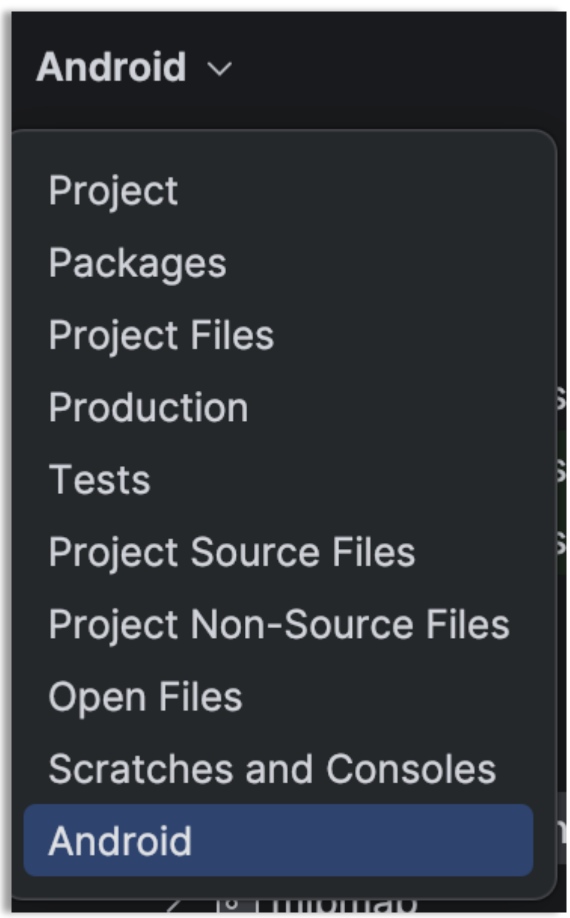


3. Configure your project
    a. Give the project a name
    b. Make sure the language is Kotlin
    c. The minimum API level should be enough so that the application runs on most devices

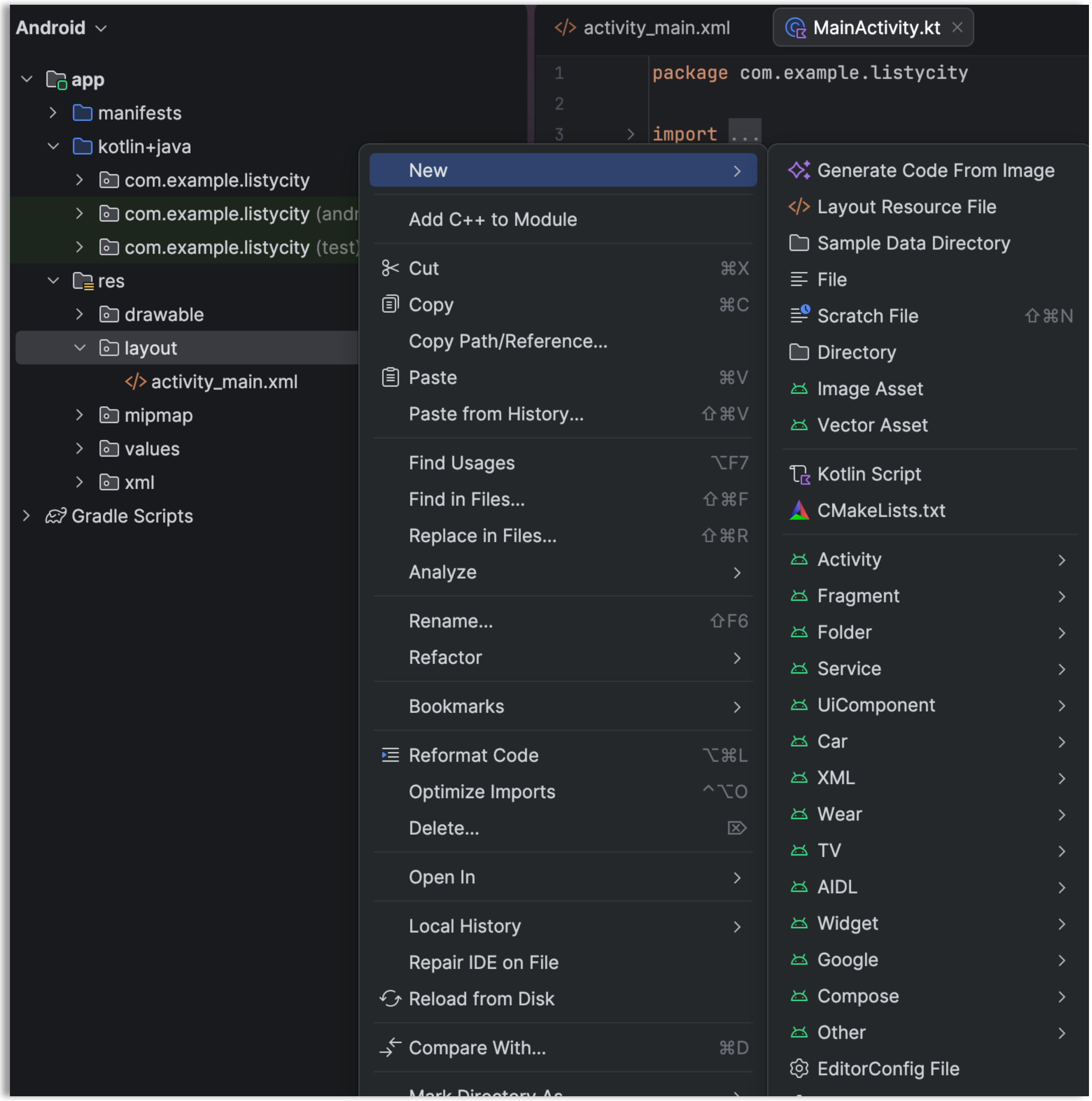
    Click Finish and wait for the project to build



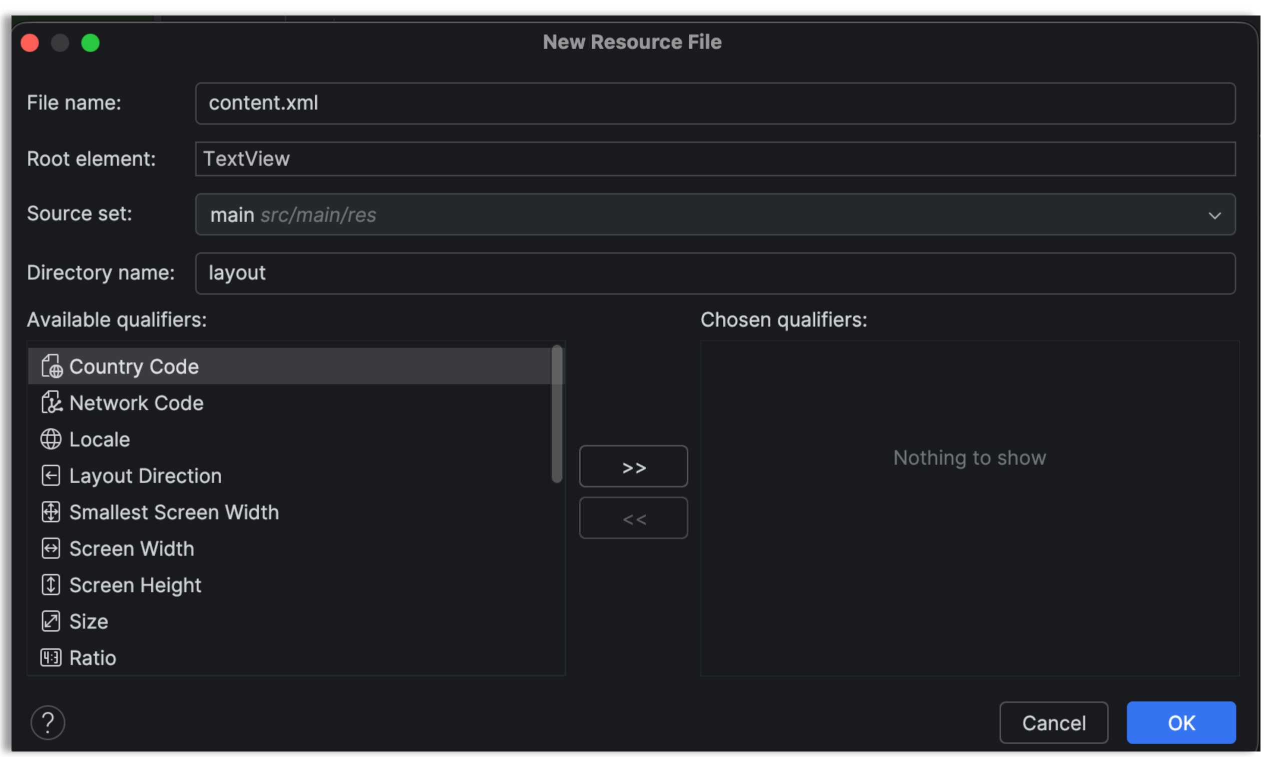
4. Ensure the navigation view is set to Android



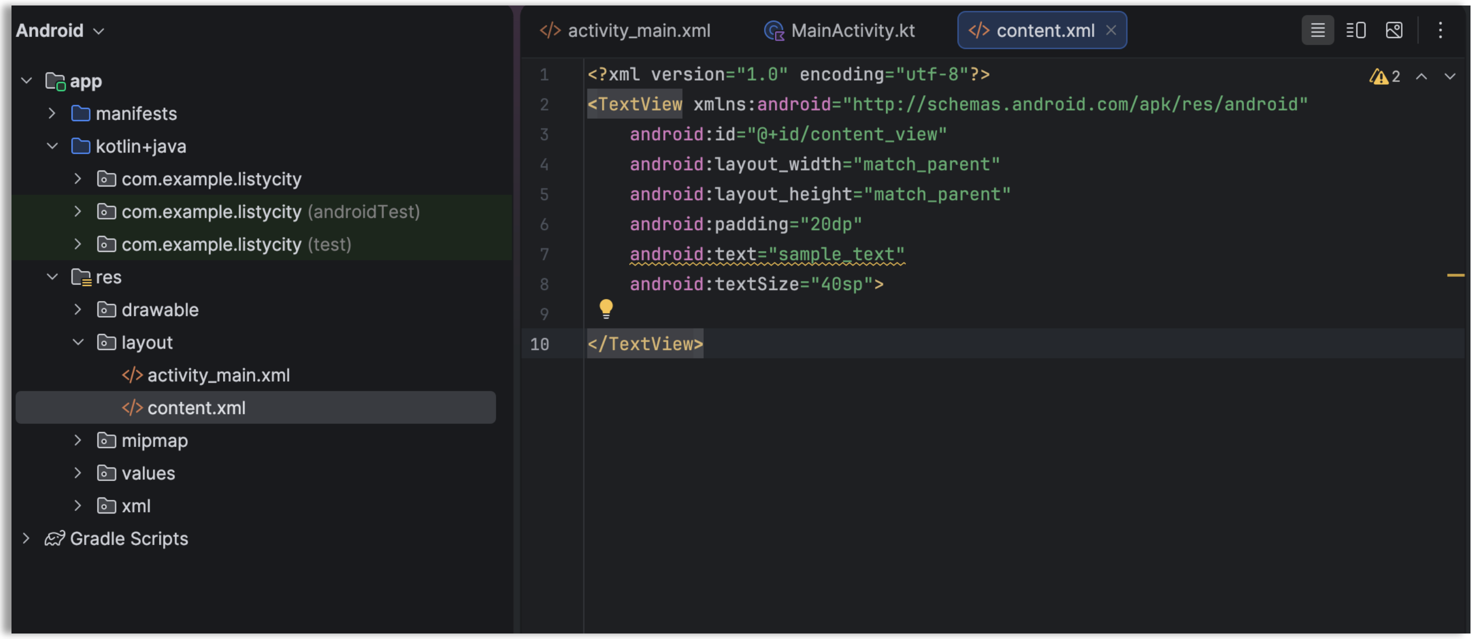
    Navigate to ‘app / res / layout directory’
    Then, right-click on the ‘layout’ directory and then click on ‘New’ and then ‘Layout Resource File’



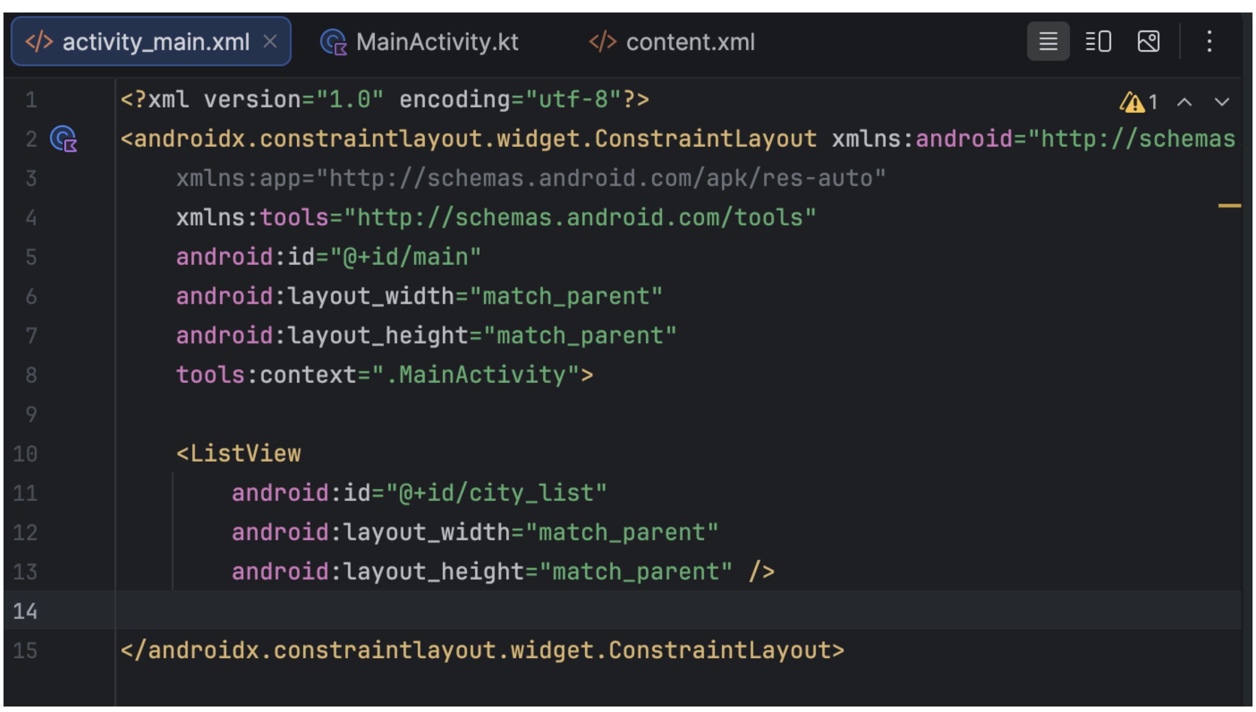
5. Give a name to the Layout Resource File with the extension ‘.xml’.
   Then, rename the “Root element” to “TextView”, and then press ‘OK’.



6. Now, go to ‘content.xml’ under ‘res / layout’ and add the following properties to TextView.
   Make sure the ‘Text’ tab is selected. Make sure you add an ‘id’ to the TextView (topmost tabbed line)



7. Go to activity_main.xml under the ‘res / layout’ directory and add a ‘ListView’ inside the ‘constraintlayout’ ViewGroup. If there is a TextView already in activity_main.xml, delete it and input the ListView.
   Make sure you add an ‘id’ attribute to the ‘ListView’



8. Now, let’s head over to ActivityMain.kt, which contains the logic for mapping the data to the ‘ListView’ so that it can be shown as a scrolling list.

9. Create References for the 'ListView' along with a reference for 'ArrayAdapter' and a mutable ArrayList for our city data

```kotlin
class MainActivity : AppCompatActivity() {  2 Usages
    // lateinit means "this will be initialized later"
    private lateinit var cityList: ListView
    // means the adapter will connect the list data to the ListView,
    // but it will be initialized after the data/list view exists
    private lateinit var cityAdapter: ArrayAdapter<String>
    // means "Create an editable list of city names"
    private val dataList = mutableListOf<String>()
```

10. Now, inside the onCreate() method, we write the logic that will help to bind the data to the 'ListView'
    a. First, find the reference to the 'ListView' using findViewById() and assign it to the reference 'cityList'
    b. Then, declare a string array consisting of cities, which can be fed into the 'ListView' later
    c. Create a new 'ArrayList' and assign it to the reference 'dataList'.
        i. This will now contain the data (the string array of cities)
    d. Add the data (string array containing city names) to the 'dataList' as shown in the picture below
    e. Now we have to link the content.xml to the 'dataList' so that each element will be displayed in a separate row in the list

```kotlin
class MainActivity : AppCompatActivity() {  2 Usages
    // lateinit means "this will be initialized later"
    private lateinit var cityList: ListView  2 Usages
    // means the adapter will connect the list data to the ListView,
    // but it will be initialized after the data/list view exists
    private lateinit var cityAdapter: ArrayAdapter<String>  2 Usages
    // means "Create an editable list of city names"
    private val dataList = mutableListOf<String>()  2 Usages

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        cityList = findViewById<ListView>(R.id.city_list)

        val cities = listOf(
            "Edmonton", "Vancouver", "Moscow",
            "Sydney", "Berlin", "Vienna",
            "Tokyo", "Beijing", "Osaka",
            "New Delhi",
        )

        dataList.addAll( elements = cities)

        cityAdapter = ArrayAdapter( context = this,  resource = R.layout.content,
             objects = dataList)
        cityList.adapter = cityAdapter

        ViewCompat.setOnApplyWindowInsetsListener( view = findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets( typeMask = WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
```